Case Study: Food Delivery Order Processing Using Executor Framework

Scenario

A food delivery application receives multiple customer orders at the same time. Each order must go through the following activities:

- Validate the order
- Confirm payment
- Notify the restaurant
- Assign a delivery partner
- Send confirmation to the customer

Processing every order using the main thread would make the application slow. Creating a separate thread manually for every order may also consume excessive system resources.

Objective

Use the Java Executor Framework to process multiple food orders efficiently using a fixed number of worker threads.

Requirements

1. Create an `OrderTask` class that implements `Runnable`.
2. Each task should contain an order number.
3. Simulate order processing by displaying:
4. Order number
5. Thread name
6. Processing status
7. Add a delay of two seconds to represent payment verification and restaurant confirmation.
8. Create a fixed thread pool containing three threads.
9. Submit six food orders to the executor.
10. Ensure that only three orders are processed simultaneously.
11. Shut down the executor after submitting all orders.

Expected Execution

The first three orders may start processing immediately:

```
Order 1 is being processed by pool-1-thread-1
Order 2 is being processed by pool-1-thread-2
Order 3 is being processed by pool-1-thread-3
```

Orders 4, 5, and 6 remain in the executor queue.

After one of the worker threads finishes processing an order, it picks the next available order:

```
Order 4 is being processed by pool-1-thread-2
Order 5 is being processed by pool-1-thread-1
Order 6 is being processed by pool-1-thread-3
```

The exact execution order may change because thread scheduling is controlled by the JVM and operating system.

Concepts Covered

- `ExecutorService`
- `Executors.newFixedThreadPool()`
- `Runnable`
- `submit()`
- Task queue
- Thread reuse
- Concurrent task execution
- `shutdown()`

Business Benefit

The Executor Framework allows the food delivery application to process several orders concurrently without creating unlimited threads. It improves performance, controls resource usage, and simplifies thread management.